Terraform-05

Task 2 — Practical Checklist

Video practicals

0 of 6 completed

1. Local-exec provisioner
2. Remote-exec provisioner and EC2 user_data
3. Provisioner create/destroy and on_failure behavior
4. Terraform taint and untaint
5. Terraform debugging using TF_LOG and TF_LOG_PATH
6. Terraform import

Practical 1 — Terraform local-exec Provisioner

Objective: Create a file using Terraform and execute a command on your Windows computer.

Step 1 — Create a new project folder

Open your Windows VS Code PowerShell:

```
cd C:\Users\HP\terraform-05
mkdir task-2a-local-exec
cd task-2a-local-exec
code .
```

Step 2 — Create main.tf

Paste this code:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}
```

```
resource "local_file" "demo" {
  filename = "${path.module}/pets.txt"
  content  = "I love cats"

  provisioner "local-exec" {
    command     = "echo Created pets.txt using Terraform local-exec > created-log.txt"
    interpreter = ["cmd", "/C"]
  }
}
```

Step 3 — Understand the code

Code	Purpose
local_file	Creates pets.txt
content	Writes text inside the file
local-exec	Runs a command on your Windows computer
command	Creates another file named created-log.txt
interpreter	Executes the command using Windows Command Prompt

Step 4 — Execute Terraform

Run these commands one by one:

```
terraform init
terraform fmt
terraform validate
terraform plan
```

Expected plan:

Plan: 1 to add, 0 to change, 0 to destroy.

Now execute:

terraform apply

Type yes when prompted.

Step 5 — Verify the practical

Get-Content .\pets.txt

Expected:

I love cats

Verify the provisioner output:

Get-Content .\created-log.txt

Expected:

Created pets.txt using Terraform local-exec

Check Terraform state:

terraform state list

Expected:

local_file.demo

Real-time DevOps explanation

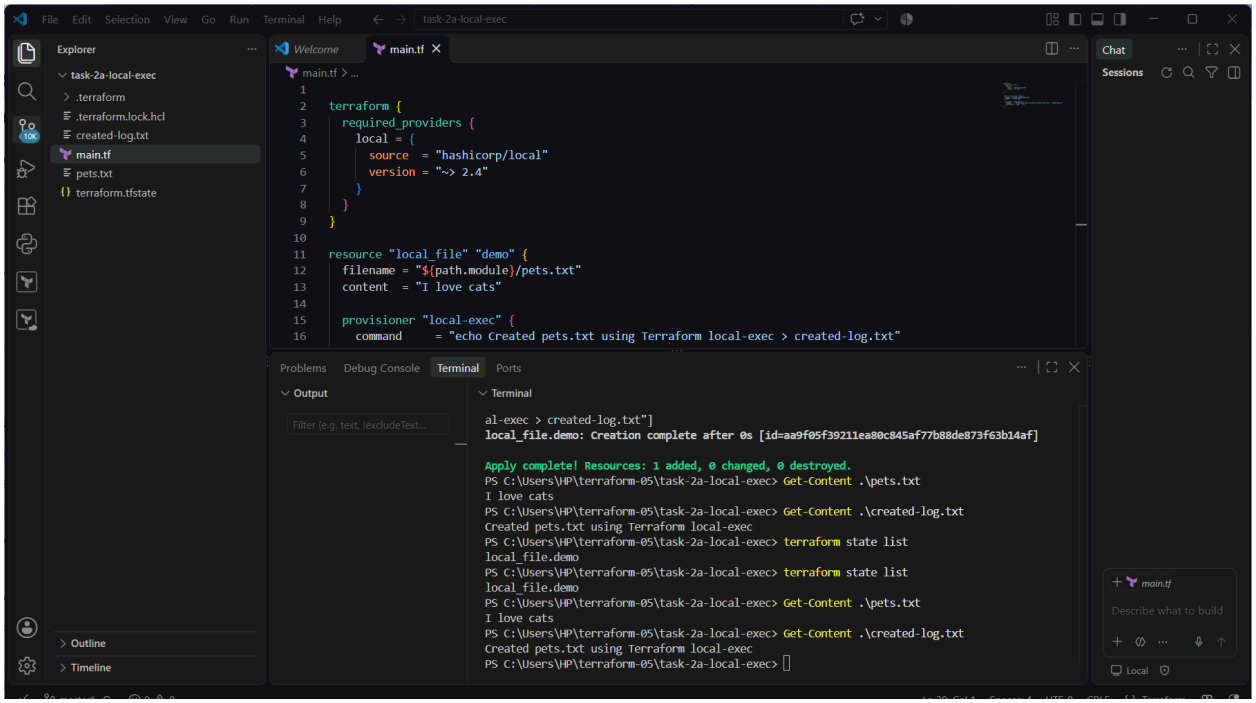
A local-exec provisioner runs commands on the machine executing Terraform, not on the EC2 instance being created.

For example, after Terraform provisions an AWS EC2 instance, local-exec can run a local script to record the instance's public IP for a lab. In production, provisioners should generally be a last resort; prefer purpose-built Terraform resources, outputs, or other automation.

Your next action: Complete Practical 1 and share these three outputs:

terraform state list

Get-Content .\pets.txt
Get-Content .\created-log.txt



Task 2B — Create-time and Destroy-time Provisioners

Objective: Understand how Terraform can execute commands when a resource is created and when it is destroyed.

We'll use local files again, so there's no need to create another paid AWS EC2 instance.

Step 1 — Create a new folder

Run in Windows PowerShell:

cd C:\Users\HP\terraform-05

mkdir task-2b-provisioner-lifecycle
cd task-2b-provisioner-lifecycle

code .

Create `main.tf` and paste:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

resource "local_file" "lifecycle_demo" {
  filename = "${path.module}/lifecycle.txt"
  content  = "Terraform-05 provisioner lifecycle demo"

  # Runs when resource is created
  provisioner "local-exec" {
    when        = create
    interpreter = ["cmd", "/C"]
    command     = "echo CREATE: Resource created >> lifecycle-log.txt"
  }

  # Runs before resource is destroyed
  provisioner "local-exec" {
    when        = destroy
    interpreter = ["cmd", "/C"]
    command     = "echo DESTROY: Resource destroyed >>
lifecycle-log.txt"
  }
}
```

Step 2 — Execute Terraform

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

Enter `yes` when Terraform requests approval.

Step 3 — Verify create-time execution

Get-Content .\lifecycle.txt
Get-Content .\lifecycle-log.txt

Expected log:

CREATE: Resource created

Step 4 — Test destroy-time execution

This destroys only the local file managed by this new project, not your AWS EC2 instance.

terraform destroy

Enter `yes`, then run:

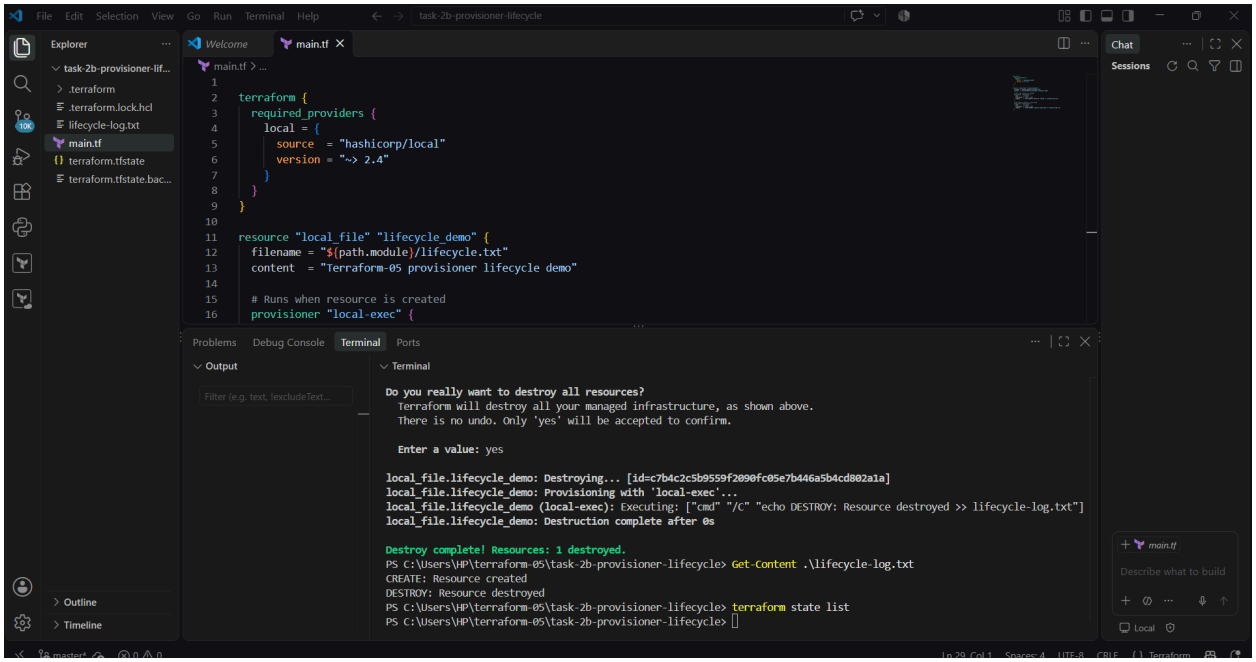
Get-Content .\lifecycle-log.txt
terraform state list

Expected log:

CREATE: Resource created
DESTROY: Resource destroyed

The Terraform state list should be empty.

Interview point: `when = create` is the default behavior, while `when = destroy` runs the provisioner before the resource is destroyed.



Task 2C — Provisioner Failure Handling (`on_failure`)

Objective:

Learn what happens when a provisioner command fails and why Terraform marks a resource as tainted.

Terraform supports two provisioner failure settings:

Setting	Behavior
`on_failure = fail`	Stops the apply and marks a newly created resource as tainted
`on_failure = continue`	Ignores the provisioner error and continues

The default is `fail`.

HashiCorp Developer

Step 1 — Create a new folder

In Windows PowerShell:

cd C:\Users\HP\terraform-05

mkdir task-2c-on-failure

cd task-2c-on-failure

code .

Step 2 — Create `main.tf`

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

resource "local_file" "failure_demo" {
```

```
  filename = "${path.module}/failure-demo.txt"
  content  = "Terraform-05 failure handling demo"

  provisioner "local-exec" {
    interpreter = ["cmd", "/C"]
    command     = "echo Simulating provisioner failure & exit /b 1"
    on_failure  = fail
  }
}
```

Here, `exit /b 1` deliberately causes the Windows command to return a failure.

Step 3 — Execute Terraform

terraform init

terraform fmt

terraform validate

terraform plan

terraform apply

Enter `yes` when prompted.

An error is expected in this practical

You should see a provisioner failure resembling:

Error: local-exec provisioner error

exit status 1

This is intentional. We are testing Terraform's behavior when a command fails.

Step 4 — Check whether Terraform marks the resource as tainted

Run:

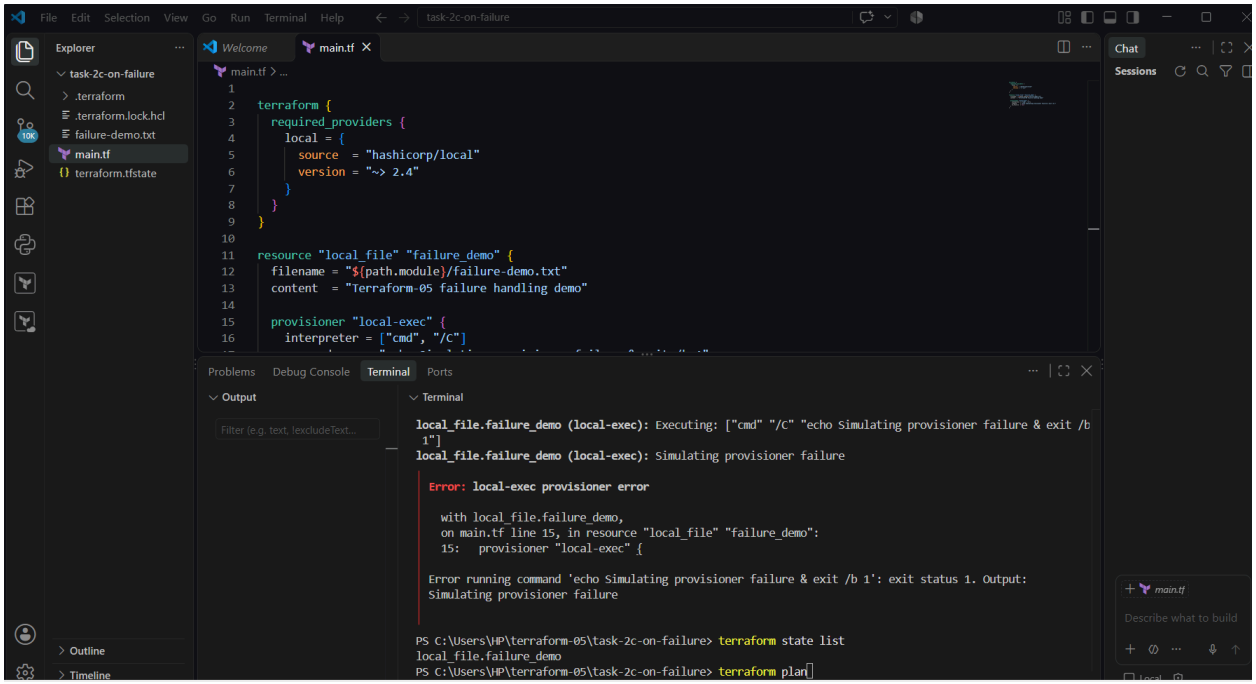
terraform state list

terraform plan

The plan should indicate that `local_file.failure_demo` must be replaced because the failed create-time provisioner left it tainted.

This helps explain why Terraform sometimes replaces a resource even when its configuration has not changed.

HashiCorp Devel



Task 2D: `on_failure = continue`

Now we'll compare `fail` and `continue` using the same local-file project.

Step 1 — Modify `main.tf`

Stay in:

C:\Users\HP\terraform-05\task-2c-on-failure

Open:

code main.tf

Find:

on_failure = fail

Change it to:

on_failure = continue

Your provisioner should now look like:

```
provisioner "local-exec" {
  interpreter = ["cmd", "/C"]
  command     = "echo Simulating provisioner failure & exit /b 1"
  on_failure  = continue
}
```

Keep the deliberately failing command unchanged. We want to test how Terraform handles the same error with a different setting.

Step 2 — Execute Terraform

terraform fmt

terraform validate

terraform plan

Since the resource is already tainted, the plan should still show:

Plan: 1 to add, 0 to change, 1 to destroy.

Now execute:

terraform apply

Type `yes`.

Step 3 — Expected result

Terraform will replace the tainted local file, run the provisioner again, and continue despite the command's nonzero exit status.

Expected:

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

The apply succeeds because we configured:

on_failure = continue

Step 4 — Verify

Run:

terraform state list

terraform plan

Get-Content .\failure-demo.txt

Expected:

local_file.failure_demo

No changes. Your infrastructure matches the configuration.

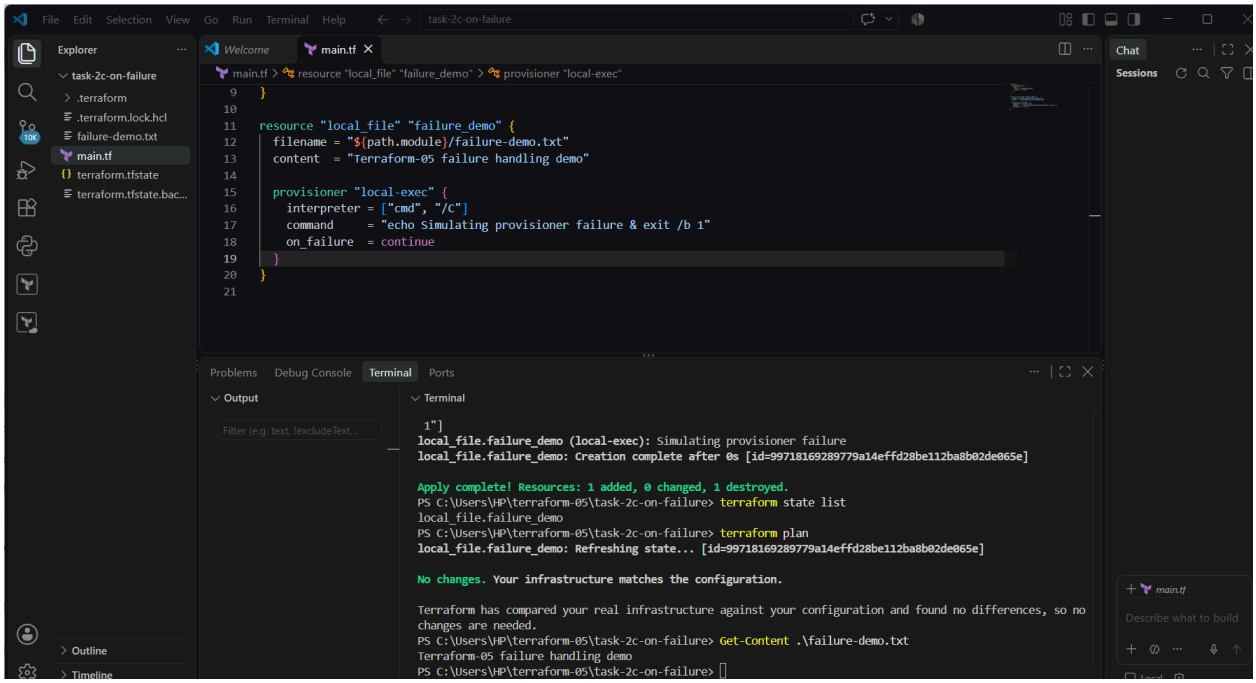
Terraform-05 failure handling demo

Important interview comparison

`on_failure = fail`	`on_failure = continue`
Default behavior	Must be explicitly configured
Stops apply on provisioner failure	Continues despite provisioner failure

Newly created resource is marked tainted	Does not taint merely because the provisioner failed
Appropriate when command success is essential	Useful only when failure is acceptable

Important: `on_failure = continue` can hide important setup failures. Avoid using it for critical installation or security configuration.



Task 2E — Terraform Taint and Untaint

Objective:

Manually mark a resource for replacement, then remove that marker without actually deleting the file.

Step 1 — Stay in your existing project

Use your Windows PowerShell:

cd C:\Users\HP\terraform-05\task-2c-on-failure

Check the resource:

terraform state list

Expected:

local_file.failure_demo

Step 2 — Mark the resource as tainted

terraform taint local_file.failure_demo

Expected:

Resource instance local_file.failure_demo has been marked as tainted.

This modifies Terraform's state to indicate that the resource should be replaced on the next apply.

Step 3 — Check the replacement plan

terraform plan

Look for:

local_file.failure_demo is tainted, so must be replaced

-/+ destroy and then create replacement

Plan: 1 to add, 0 to change, 1 to destroy.

Do not apply this plan. We want to demonstrate how to cancel an unnecessary replacement.

Step 4 — Remove the tainted status

terraform untaint local_file.failure_demo

Expected:

Resource instance local_file.failure_demo has been successfully untainted.

Then run:

terraform plan

Expected:

No changes. Your infrastructure matches the configuration.

The file remains in place because we removed the tainted marker before applying the replacement.

HashiCorp Developer

Step 5 — Learn the modern replacement method

`terraform taint` is deprecated. HashiCorp recommends the `-replace` option instead.

HashiCorp Developer

To preview a replacement without marking the state as tainted:

terraform plan -replace=local_file.failure_demo

This should show a proposed replacement. It does not change the resource or mark it tainted.

Interview comparison

Command	Purpose
`terraform taint`	Legacy command to mark a resource for replacement
`terraform untaint`	Remove an existing tainted marker
`terraform plan -replace=...`	Preview a forced resource replacement
`terraform apply -replace=...`	Perform a reviewed resource replacement

Next checkpoint: Complete Steps 2–4 and share the output of:

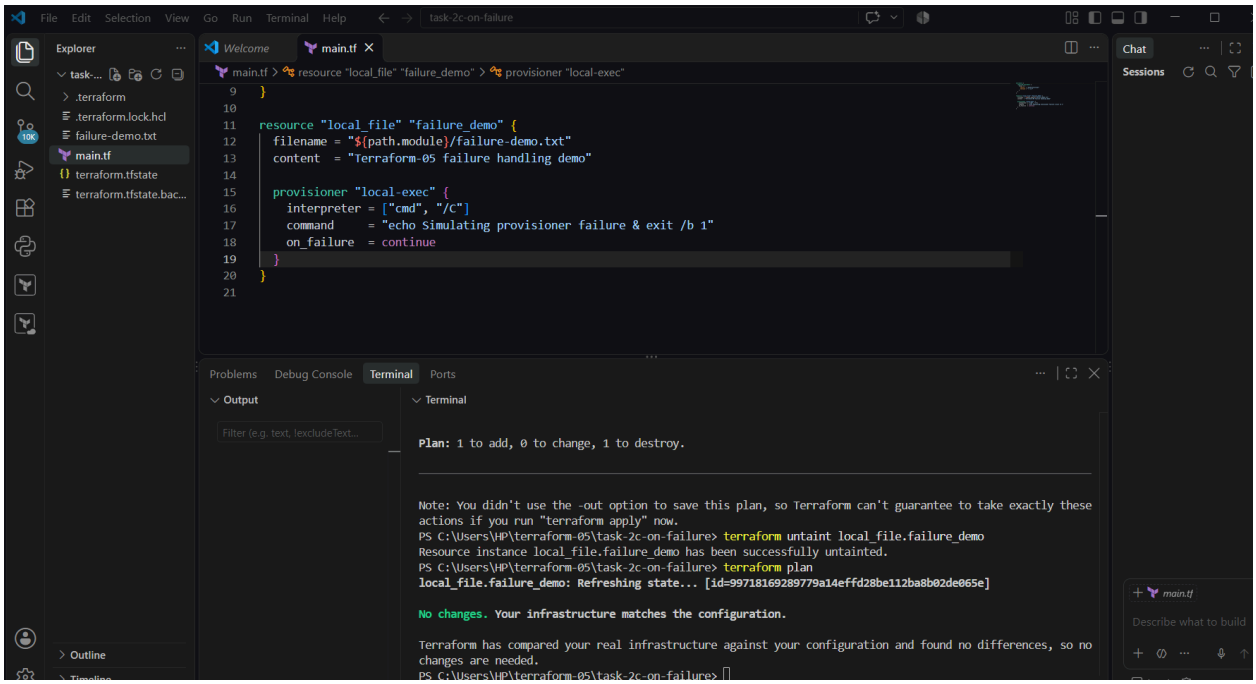
terraform taint local_file.failure_demo

terraform plan

terraform untaint local_file.failure_demo

terraform plan

Once verified, we'll move to Task 2F — Terraform Debugging with `TF_LOG` and `TF_LOG_PATH`.



Task 2F: Terraform Debugging

Objective:

Learn how to enable detailed Terraform logs and save them to a file for troubleshooting.

We'll use your existing task-2c-on-failure project, without creating any new AWS infrastructure.

Step 1 — Enable detailed logging

Run in Windows PowerShell:

cd C:\Users\HP\terraform-05\task-2c-on-failure

$env:TF_LOG = "TRACE"

TRACE is the most detailed Terraform logging level.

Step 2 — Save logs to a file

$env:TF_LOG_PATH = "PWD\terraform-debug.log"

Now run:

terraform plan

Step 3 — Verify the log file

Get-Item .\terraform-debug.log

Then:

Get-Content .\terraform-debug.log -Tail 20

You should see detailed Terraform diagnostic messages, which can include provider initialization, state operations and resource refresh information.

Step 4 — Learn the five log levels

Log level	Purpose
`TRACE`	Most detailed troubleshooting
`DEBUG`	Detailed debugging
`INFO`	Informational messages
`WARN`	Warning messages
`ERROR`	Error messages

Step 5 — Disable debugging

After completing the exercise:

Remove-Item Env:\TF_LOG

Remove-Item Env:\TF_LOG_PATH

Verify:

Test-Path Env:\TF_LOG

Test-Path Env:\TF_LOG_PATH

Expected:

False

False

Security note: TRACE logs can contain sensitive data. Don't commit them to GitHub or share the complete logs publicly.

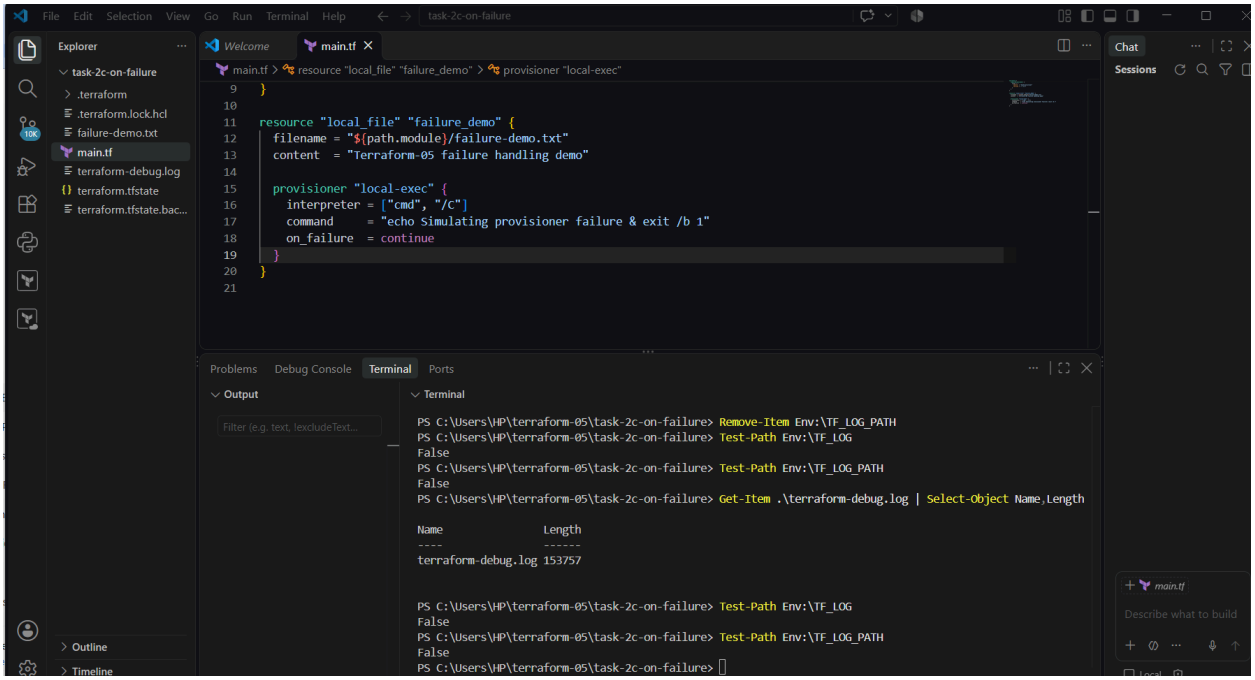
Your next checkpoint

Complete the debugging exercise and share only:

Get-Item .\terraform-debug.log | Select-Object Name,Length

Test-Path Env:\TF_LOG

Test-Path Env:\TF_LOG_PATH



ask 2G — Terraform Import Practical

Objective:

Import an existing AWS resource into Terraform state without creating that resource again.

Your trainer demonstrated importing an EC2 instance. We'll practice the same concept using a new IAM user, because your existing HTTPD EC2 is already managed by Terraform and should not be imported into another state file.

Step 1 — Create a separate project

Run in Windows PowerShell:

cd C:\Users\HP\terraform-05

mkdir task-2g-import

cd task-2g-import

code .

Step 2 — Create an AWS IAM user manually

First, configure your AWS profile:

$env:AWS_PROFILE = "terraform-lab"

Create a demonstration IAM user using AWS CLI:

aws iam create-user `

  --user-name terraform05-import-demo-986881030460 `

  --profile terraform-lab

This creates an IAM user outside Terraform. We won't create any access keys, console passwords or permissions for this user.

Verify:

aws iam get-user `

  --user-name terraform05-import-demo-986881030460 `

  --profile terraform-lab

Step 3 — Create `main.tf`

Paste the following code:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
```

```
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_iam_user" "import_demo" {
  name = "terraform05-import-demo-986881030460"
  path = "/"
}
```

We defined the resource configuration, but we haven't asked Terraform to create it.

Step 4 — Initialize Terraform

terraform init

terraform validate

Step 5 — Import the existing IAM user

Run:

terraform import aws_iam_user.import_demo terraform05-import-demo-986881030460

Expected:

aws_iam_user.import_demo: Importing from ID ...

aws_iam_user.import_demo: Import prepared!

aws_iam_user.import_demo: Refreshing state...

Import successful!

The syntax is:

terraform import RESOURCE_ADDRESS EXISTING_RESOURCE_ID

For IAM users, the import ID is the username. The CLI import command adds the existing resource to Terraform state; it does not create a new AWS user.

HashiCorp Developer

+1

Step 6 — Verify the import

terraform state list

Expected:

aws_iam_user.import_demo

Then:

terraform state show aws_iam_user.import_demo

Finally:

terraform plan

Expected if the configuration matches the existing user:

No changes. Your infrastructure matches the configuration.

This demonstrates that Terraform can begin managing infrastructure that was originally created outside Terraform.

Important: Don't run `terraform apply` before importing the IAM user. Also, don't import your existing HTTPD EC2 instance into this separate project because Terraform already manages it under Task 3.

Once the import and verification are finished, we can remove this demonstration IAM user safely with `terraform destroy` from the `task-2g-import` folder.

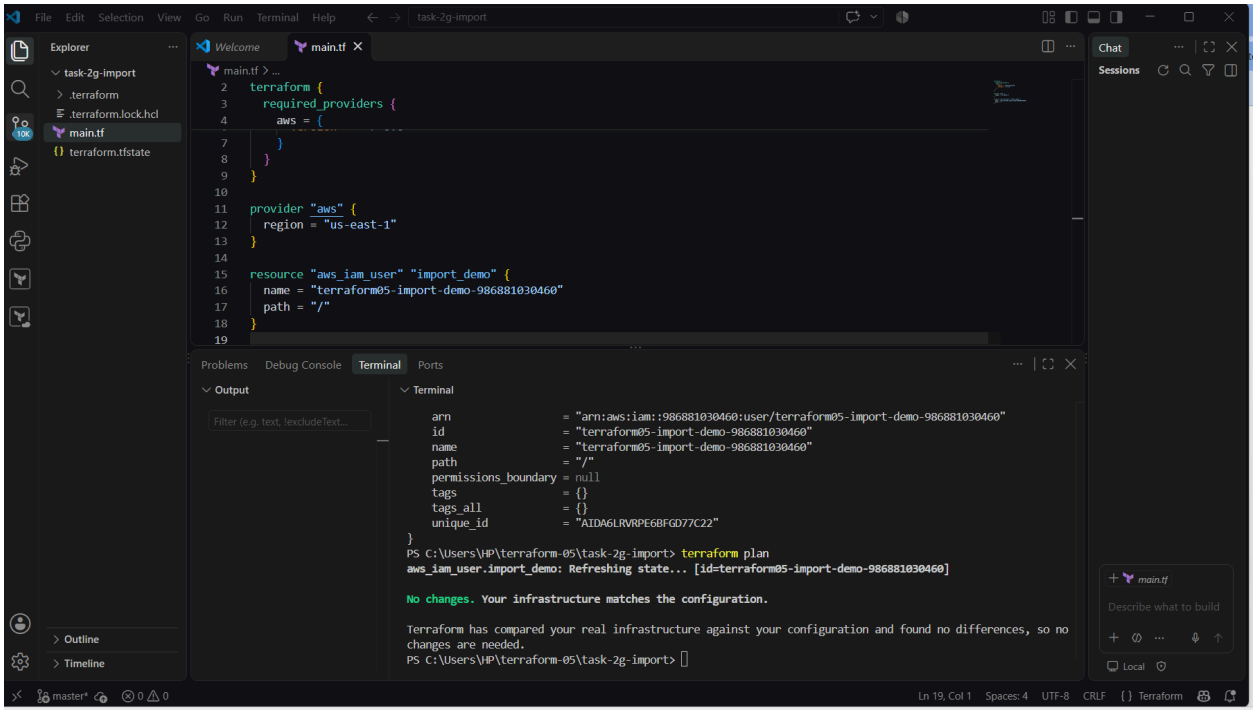
Your next checkpoint

Complete the import and share the outputs of:

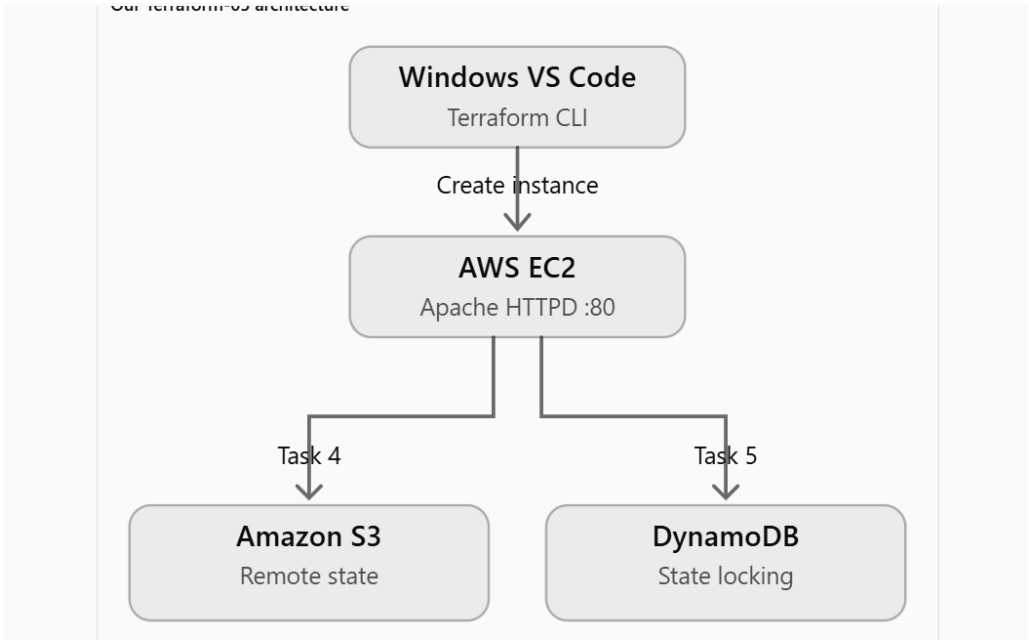
terraform import aws_iam_user.import_demo terraform05-import-demo-986881030460

terraform state list

terraform plan



3.Create one EC2 instance with httpd installed using a Terraform script.



Step 1 — Create a new folder in VS Code

Open Windows PowerShell:

cd C:\Users\HP

mkdir terraform-05
cd terraform-05

mkdir task-3-httpd
cd task-3-httpd

code .

Create a file named `main.tf`.

Step 2 — Add Terraform configuration

Copy this code into `main.tf`:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Use the latest Amazon Linux 2023 AMI
data "aws_ssm_parameter" "amazon_linux" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

# Find default VPC
data "aws_vpc" "default" {
  default = true
}
```

```
# Find subnets in the default VPC
data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# Security group for HTTP traffic
resource "aws_security_group" "httpd_sg" {
  name_prefix = "terraform05-httpd-"
  description = "HTTP access for Terraform 05 lab"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Terraform05-HTTPD-SG"
  }
}

# EC2 instance with Apache HTTPD
resource "aws_instance" "httpd_server" {
  ami           = data.aws_ssm_parameter.amazon_linux.value
  instance_type = "t3.micro"

  subnet_id                   = data.aws_subnets.default.ids[0]
  associate_public_ip_address = true

  vpc_security_group_ids = [
    aws_security_group.httpd_sg.id
  ]
```

```
  user_data = <<-EOF
    #!/bin/bash
    set -e
    dnf install -y httpd

    echo "<h1>Terraform-05 HTTPD Server Working!</h1>" > /var/www/html/index.html

    systemctl enable --now httpd
  EOF

  tags = {
    Name = "Terraform05-HTTPD-Server"
  }
}

output "instance_id" {
  value = aws_instance.httpd_server.id
}

output "public_ip" {
  value = aws_instance.httpd_server.public_ip
}

output "website_url" {
  value = "http://${aws_instance.httpd_server.public_ip}"
}
```

This example uses EC2 `user_data` to install Apache during first boot, avoiding SSH provisioners. It uses a default VPC and opens HTTP port 80 publicly for a temporary demonstration; restrict access when the lab is finished.

Step 3 — Verify your AWS credentials

In Windows PowerShell:

```
$env:AWS_PROFILE = "terraform-lab"

aws sts get-caller-identity

terraform version
```

Make sure the AWS account shown is the one where you want the EC2 instance created.

Step 4 — Execute Terraform

Run the commands in order:

terraform init
terraform fmt
terraform validate
terraform plan

Expected plan:

Plan: 2 to add, 0 to change, 0 to destroy.

The two resources are the EC2 instance and Security Group. The AWS data sources only read existing information.

Review the plan, then run:

terraform apply

Type `yes` when prompted.

Step 5 — Verify HTTPD

After Terraform completes:

terraform output

Terraform output confirms:

Terraform-05 HTTPD Server

EC2 Created

Setting	Value
Instance ID	`i-0de27553fcc7fde66`
Public IP	`54.167.221.156`

Region	`us-east-1`
Resources created	2
Terraform apply	Successful

We still need to verify Apache HTTPD is serving the webpage. Open:

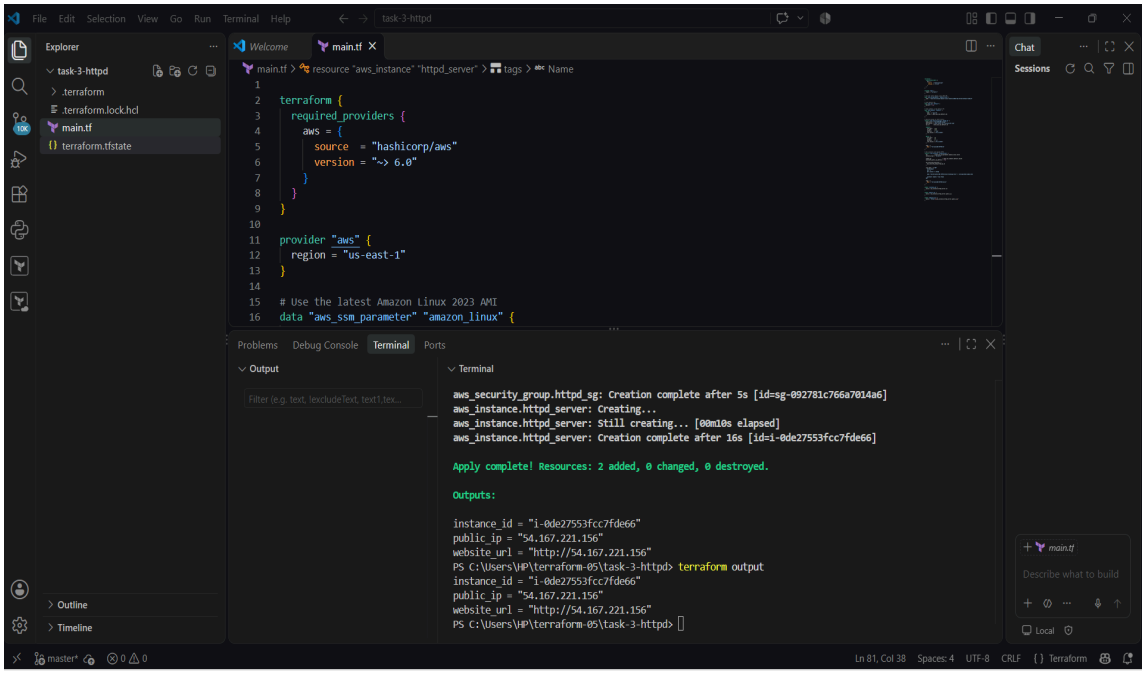
[Open Terraform-05 HTTPD webpage]

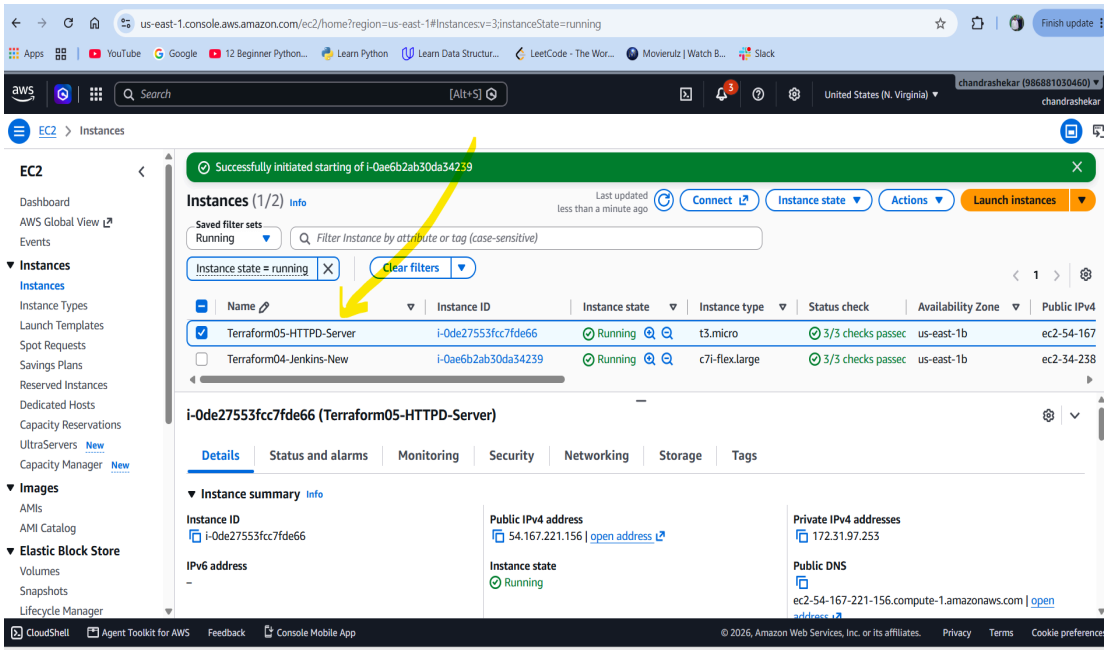
Expected message:

> Terraform-05 HTTPD Server Working!

You can also verify from Windows PowerShell:

```
Invoke-WebRequest -Uri "http://54.167.221.156" -UseBasicParsing |
  Select-Object -ExpandProperty Content
```





4). Set up S3 as backend for task 3.

Task 4: Configure AWS S3 as Terraform Remote Backend

Date: 28 October 2026
Project: Terraform-05
Task: Configure Amazon S3 as the remote backend for the HTTPD EC2 instance created in Task 3.
Status: ✅ Successfully completed and verified

---

1. Objective

The objective of this task is to store the Terraform state file in an Amazon S3 bucket instead of storing it only on a local machine.

This allows multiple DevOps engineers and CI/CD pipelines to access a centralized Terraform state file.

Why do we need a remote backend?

By default, Terraform stores infrastructure information in a local file named `terraform.tfstate`.

Local state has limitations:

- It is difficult to share safely between team members.
- Losing the local state file can cause infrastructure management problems.
- Different developers may accidentally use outdated state.
- It is unsuitable for reliable team-based infrastructure automation.

Using an S3 backend provides centralized state storage, and S3 versioning can help recover previous versions of the state file.

2. Prerequisites

Before starting this task, I completed Task 3 and created an AWS EC2 instance using Terraform.

Existing Task 3 infrastructure:

Component	Details
AWS Region	us-east-1
EC2 Instance ID	`i-0de27553fcc7fde66`
EC2 Public IP	`54.167.221.156`
Security Group	`sg-092781c766a7014a6`
AWS CLI Profile	`terraform-lab`
Terraform Project	`task-3-httpd`

Required tools:

- Terraform CLI
- AWS CLI
- VS Code
- AWS credentials with required S3 permissions
- Windows PowerShell

3. Create a Separate Backend Folder

Opened Windows PowerShell and executed:

cd C:\Users\HP\terraform-05

mkdir task-4-backend-bootstrap

cd task-4-backend-bootstrap

code .

Created a Terraform configuration file named:

main.tf

Reason: The S3 bucket should be managed separately from the EC2 infrastructure. This prevents the backend bucket from being destroyed accidentally when the HTTPD EC2 instance is removed.

4. Write Terraform Code to Create the S3 Bucket

Added the following configuration to `main.tf`:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

data "aws_caller_identity" "current" {}

resource "aws_s3_bucket" "terraform_state" {
  bucket_prefix = "terraform05-${data.aws_caller_identity.current.account_id}-"

  lifecycle {
    prevent_destroy = true
  }

  tags = {
    Name    = "Terraform05-Remote-State"
```

```
    Project = "Terraform-05"
  }
}

resource "aws_s3_bucket_versioning" "state_versioning" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_public_access_block" "state_security" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_server_side_encryption_configuration" "state_encryption" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

output "state_bucket_name" {
  value = aws_s3_bucket.terraform_state.id
}
```

Explanation of the Resources

Terraform Resource	Purpose
`aws_s3_bucket`	Creates the remote state bucket
`aws_s3_bucket_versioning`	Enables state-file version history

`aws_s3_bucket_public_access_block`	Blocks public access to the bucket
`aws_s3_bucket_server_side_encryption_configuration`	Enables server-side encryption

Also used `prevent_destroy = true` to protect the bucket against accidental destruction through Terraform.

5. Initialize and Execute Terraform

First, configured the AWS CLI profile:

$env:AWS_PROFILE = "terraform-lab"

Initialized Terraform:

terraform init

Formatted the Terraform code:

terraform fmt

Validated the configuration:

terraform validate

Checked the execution plan:

terraform plan

The intended plan was to create four S3-related Terraform resources.

After reviewing the changes:

terraform apply

Entered `yes` to approve the changes.

Retrieved the bucket name:

terraform output -raw state_bucket_name

S3 Bucket Used

terraform05-986881030460-fb58b087846653c9a85ffb6795

6. Back Up the Existing Terraform State

Navigated to the existing Task 3 project:

cd C:\Users\HP\terraform-05\task-3-httpd

Before migrating the state, the recommended backup command was:

Copy-Item .\terraform.tfstate .\terraform.tfstate.backup-before-s3

This backup protects the existing state in case the migration fails.

Important: Terraform state files may contain sensitive infrastructure information and should not be committed to a public GitHub repository.

7. Create the Backend Configuration File

In the `task-3-httpd` directory, created a new file named:

backend.tf

Added this configuration:

```
terraform {
  backend "s3" {
    bucket  = "terraform05-986881030460-fb58b087846653c9a85ffb6795"
    key     = "terraform05/task3/terraform.tfstate"
    region  = "us-east-1"
    profile = "terraform-lab"
    encrypt = true
  }
}
```

Explanation of Backend Attributes

Attribute	Explanation
`backend "s3"`	Uses Amazon S3 for Terraform state storage
`bucket`	Specifies the S3 bucket name
`key`	Specifies the state object's path inside S3
`region`	AWS region containing the bucket

`profile`	AWS CLI profile used for authentication
`encrypt`	Enables encryption for the state object

The `key` does not create a local folder. It specifies the S3 object key where the Terraform state will be stored.

8. Migrate the Local State to Amazon S3

Executed:

terraform init -migrate-state

Terraform initialized the S3 backend and transferred the existing local state to the remote backend.

If Terraform prompts for confirmation to copy the existing state, enter:

yes

This migration allows the same Terraform configuration to continue managing the existing EC2 instance without recreating it.

9. Verify the S3 State File

Executed:

aws s3 ls s3://terraform05-986881030460-fb58b087846653c9a85ffb6795/terraform05/task3/ --profile terraform-lab

Actual output:

2026-10-28 16:16:36 12208 terraform.tfstate

This confirms that the Terraform state object exists in S3.

Verification result: ✅ Passed

10. Verify the Active Terraform Backend

Executed these PowerShell commands:

$meta = Get-Content .\.terraform\terraform.tfstate -Raw | ConvertFrom-Json

$meta.backend.type

$meta.backend.config.key

Actual output:

s3
terraform05/task3/terraform.tfstate

This confirms that Terraform is configured to use the S3 backend with the expected state key.

Verification result: ✅ Passed

11. Verify That Existing Infrastructure Is Unchanged

Executed:

terraform plan

Actual output:

aws_instance.httpd_server: Refreshing state...
[id=i-0de27553fcc7fde66]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure
against your configuration and found no differences,
so no changes are needed.

This confirmed that migrating the state to S3 did not require changes to the EC2 infrastructure.

Verification result: ✅ Passed

12. Final Folder Structure

C:\Users\HP\terraform-05\
│
├── task-3-httpd\
│ ├── main.tf
│ ├── backend.tf
│ ├── .terraform\
│ └── local state backup (if created)
│
└── task-4-backend-bootstrap\

```
├── main.tf
├── terraform.tfstate
└── .terraform\
```

The Task 3 working directory retains backend metadata locally, while the active infrastructure state is stored remotely in S3.

13. Troubleshooting Encountered

Issue 1 — S3 Bucket Not Found

Error:

NoSuchBucket: The specified bucket does not exist

Reason: The command was initially executed using the placeholder `YOUR-ACTUAL-BUCKET-NAME`.

Solution: Replaced the placeholder with the actual generated S3 bucket name.

Issue 2 — S3 Listing Returned No Objects

The S3 listing initially returned no state file.

Investigation: Checked whether `backend.tf` existed.

The command:

Get-Content .\backend.tf

returned:

Cannot find path ...\backend.tf because it does not exist.

Solution: Created `backend.tf`, configured the correct bucket and key, and initialized the backend migration.

Issue 3 — Confirming the State Was Truly Remote

`terraform plan` returned "No changes," but this alone did not prove remote state was configured.

Solution: Verified both:

1. The `terraform.tfstate` object existed inside S3.

2. Local Terraform backend metadata reported backend type `s3`.

These checks confirmed that migration had succeeded.

14. Real-Time DevOps Scenario

Consider a DevOps team with three engineers managing the same AWS infrastructure.

With local state, each engineer could have a different version of `terraform.tfstate`.

Using the S3 backend:

- Engineer A executes `terraform apply`.
- Terraform reads the shared state from S3.
- After a successful change, Terraform updates the remote state.
- Engineer B can subsequently use the same backend to access the latest recorded infrastructure state.

For safe concurrent operations, a state locking mechanism is also required. That is covered separately in Terraform-05 Task 5.

15. Interview Questions and Answers

Q1. What is Terraform remote state?

Remote state is the storage of Terraform's infrastructure state in a remote backend such as Amazon S3 rather than only on a developer's local machine.

Q2. Why do we use S3 as a Terraform backend?

We use S3 for centralized state storage, team collaboration, access control, encryption and state versioning.

Q3. What is the purpose of `terraform init -migrate-state`?

It initializes the updated backend configuration and migrates existing state from the previous backend to the new backend.

Q4. What is the difference between `main.tf` and `terraform.tfstate`?

`main.tf` contains the desired infrastructure configuration, while `terraform.tfstate` records information about the infrastructure Terraform manages.

Q5. Why do we enable S3 versioning?

Versioning retains previous state object versions, which can help recover from accidental changes or state-file corruption.

Q6. What does the `key` attribute mean in an S3 backend?

It identifies the object path where Terraform stores the state file inside the S3 bucket.

Q7. Does S3 automatically prevent two engineers from running Terraform simultaneously?

No. Centralized state storage alone is not sufficient for concurrency protection. Terraform should use a supported backend locking mechanism.

Q8. How did you verify your Terraform state migration?

I checked the state object in S3 using AWS CLI, confirmed that the initialized backend type was `s3`, and executed `terraform plan` to verify that the existing infrastructure remained unchanged.

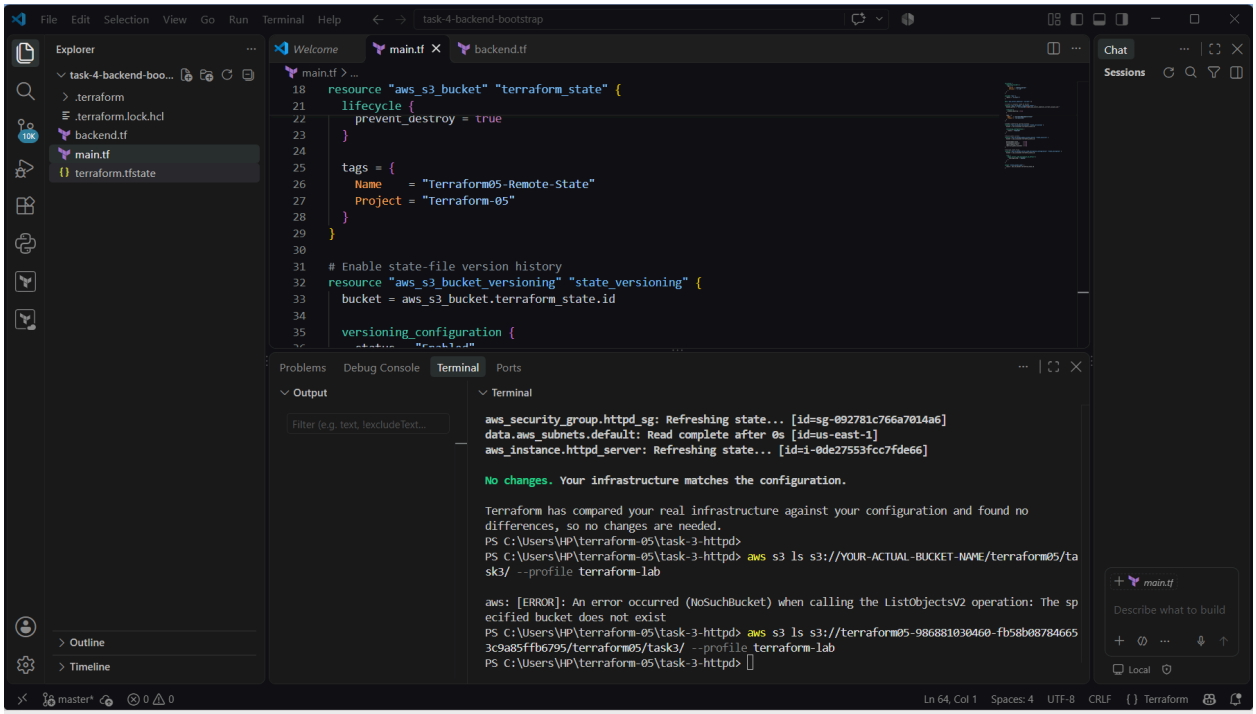
16. Final Result

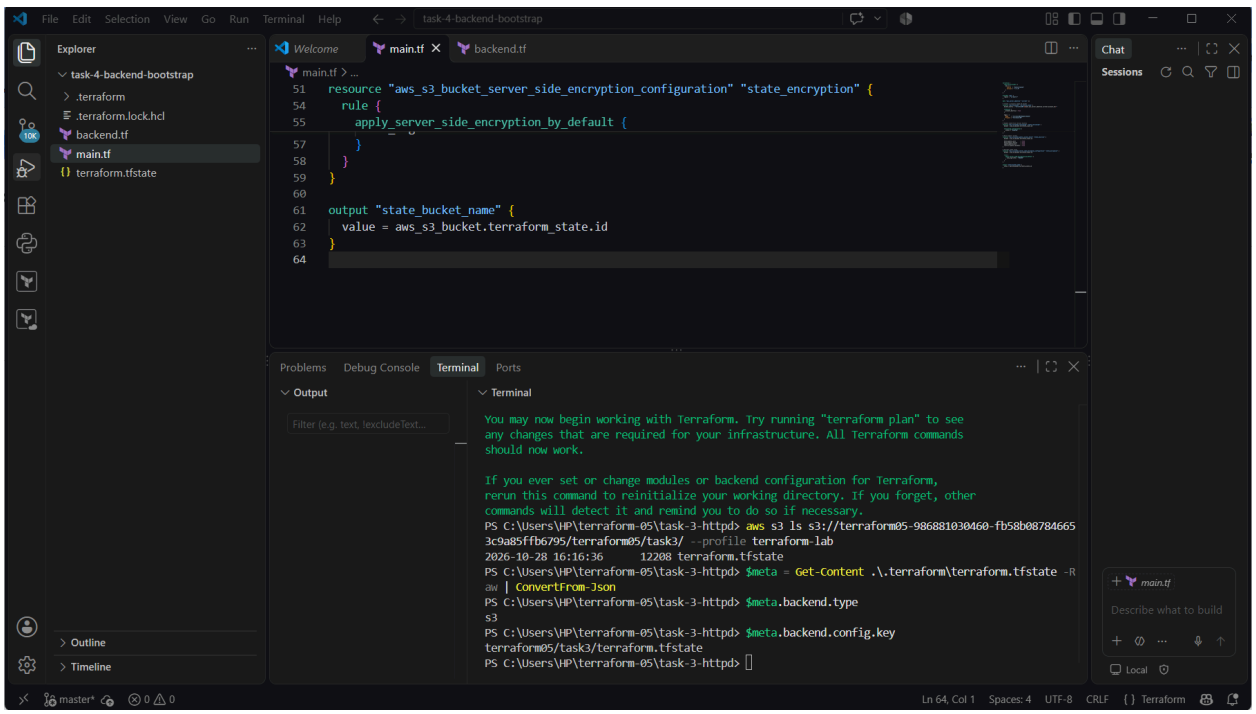
Terraform-05 Task 4: Successfully Completed ✅

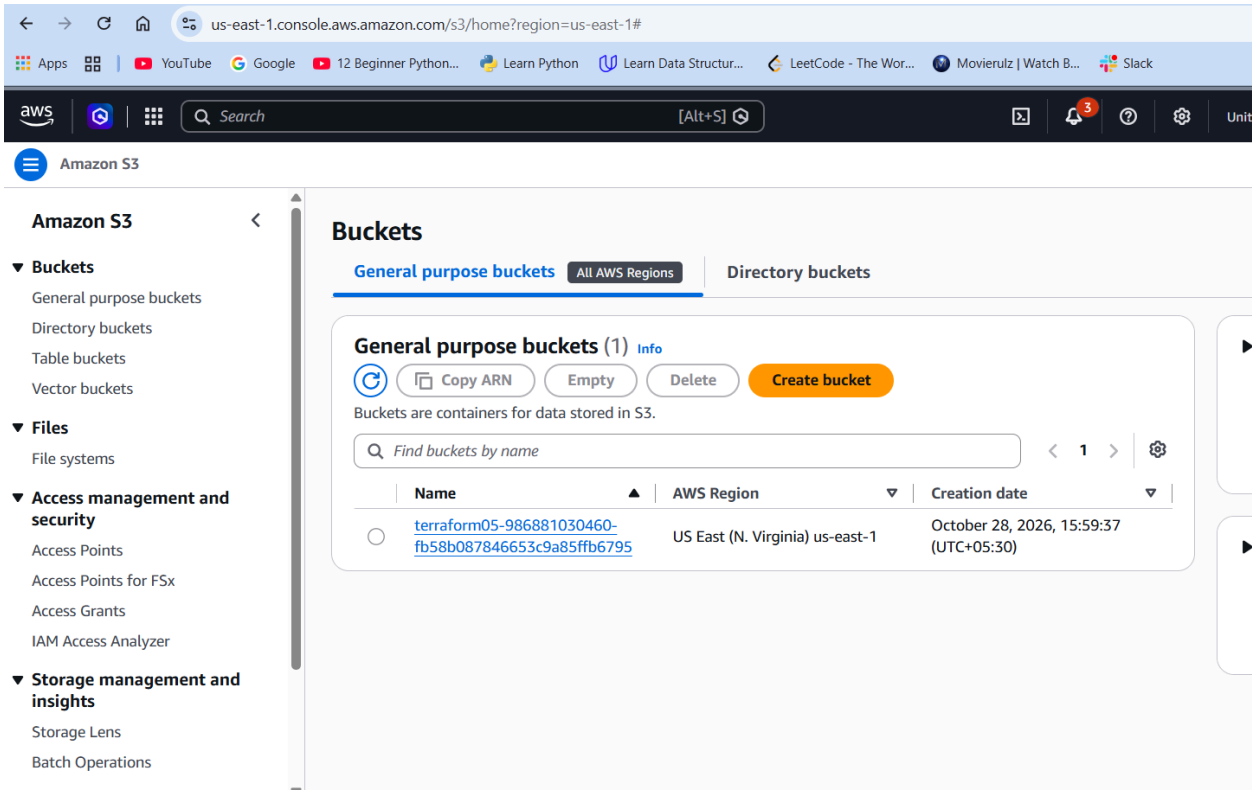
- Amazon S3 backend bucket created.
- Terraform configuration prepared for bucket versioning, public-access blocking and server-side encryption.
- Existing Task 3 state migrated into the S3 backend.
- Remote `terraform.tfstate` object verified using AWS CLI.
- Backend type and state key verified.
- Existing EC2 infrastructure remained unchanged.

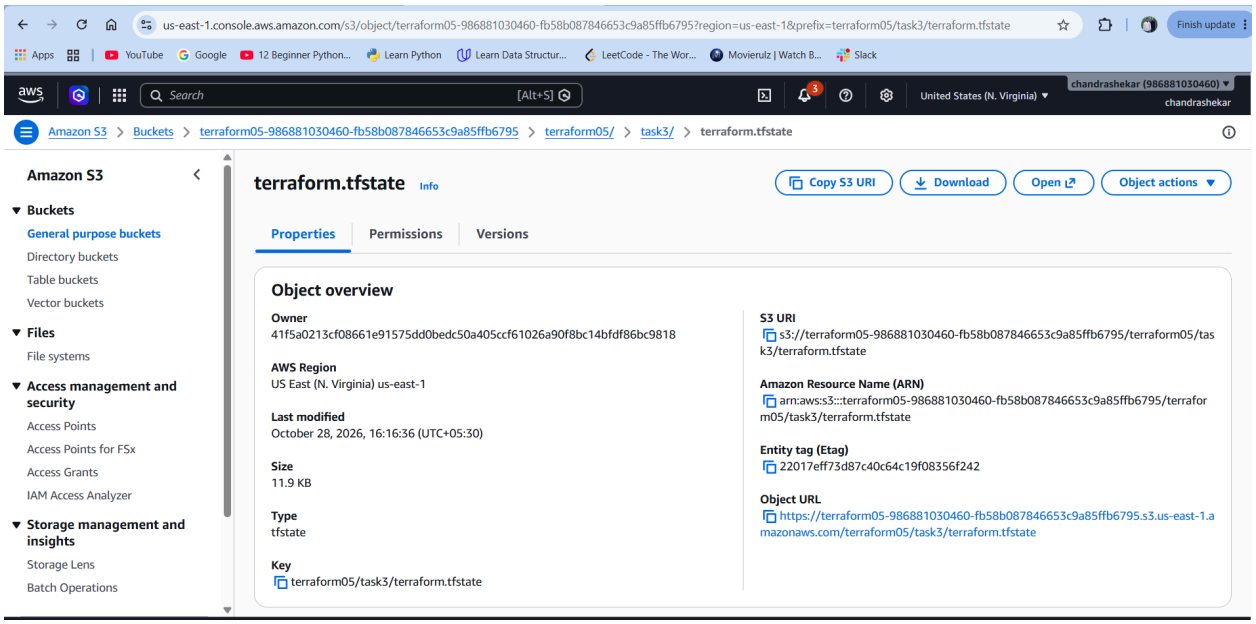
Next Task: Terraform-05 Task 5 — Configure DynamoDB State Locking.

Conclusion: Successfully configured Amazon S3 as a remote backend for Terraform state management. This practical demonstrated how to migrate existing infrastructure state from a local backend to centralized remote storage without recreating the AWS resources.


Explorer
task-4-backend-boo...
.terraform
.terraform.lock.hcl
backend.tf
main.tf
terraform.tfstate
Welcome
main.tf
backend.tf
main.tf > ...
resource "aws_s3_bucket" "terraform_state" {
lifecycle {
prevent_destroy = true
}
tags = {
Name = "Terraform05-Remote-State"
Project = "Terraform-05"
}
}
Enable state-file version history
resource "aws_s3_bucket_versioning" "state_versioning" {
bucket = aws_s3_bucket.terraform_state.id
versioning_configuration {
Chat
Sessions
Problems
Debug Console
Terminal
Ports
Output
Terminal
aws_security_group.httpd_sg: Refreshing state... [id=sg-092781c766a7014a6]
data.aws_subnets.default: Read complete after 0s [id=us-east-1]
aws_instance.httpd_server: Refreshing state... [id=i-0de27553fcc7fde66]
No changes. Your infrastructure matches the configuration.
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\Users\HP\terraform-05\task-3-httpd>
PS C:\Users\HP\terraform-05\task-3-httpd> aws s3 ls s3://YOUR-ACTUAL-BUCKET-NAME/terraform05/task3/ --profile terraform-lab
aws: [ERROR]: An error occurred (NoSuchBucket) when calling the ListObjectsV2 operation: The specified bucket does not exist
PS C:\Users\HP\terraform-05\task-3-httpd> aws s3 ls s3://terraform05-986881030460-fb58b08784665 3c9a85ffb6795/terraform05/task3/ --profile terraform-lab
PS C:\Users\HP\terraform-05\task-3-httpd>
Outline
Timeline
master*
Ln 64, Col 1
Spaces: 4
UTF-8
CRLF
Terraform


Explorer
task-4-backend-bootstrap
.terraform
.terraform.lock.hcl
backend.tf
main.tf
terraform.tfstate
Welcome
main.tf
backend.tf
main.tf > ...
resource "aws_s3_bucket_server_side_encryption_configuration" "state_encryption" {
rule {
apply_server_side_encryption_by_default {
}
}
}
output "state_bucket_name" {
value = aws_s3_bucket.terraform_state.id
}
Chat
Sessions
Problems
Debug Console
Terminal
Ports
Output
Terminal
You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.
If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.
PS C:\Users\HP\terraform-05\task-3-httpd> aws s3 ls s3://terraform05-986881030460-fb58b08784665 3c9a85ffb6795/terraform05/task3/ --profile terraform-lab
2026-10-28 16:16:36 12208 terraform.tfstate
PS C:\Users\HP\terraform-05\task-3-httpd> $meta = Get-Content .\.terraform\terraform.tfstate -Raw | ConvertFrom-Json
PS C:\Users\HP\terraform-05\task-3-httpd> $meta.backend.type
s3
PS C:\Users\HP\terraform-05\task-3-httpd> $meta.backend.config.key
terraform05/task3/terraform.tfstate
PS C:\Users\HP\terraform-05\task-3-httpd>
Outline
Timeline
master*
Ln 64, Col 1
Spaces: 4
UTF-8
CRLF
Terraform





5). Set up DynamoDB locking for task 3.

Task 5: Configure DynamoDB State Locking

Objective

The objective of this task is to create an Amazon DynamoDB table using Terraform and configure it as a state-locking mechanism for our existing S3 remote backend.

This prevents multiple Terraform operations from modifying the same infrastructure state simultaneously.

Why do we need state locking?

Consider a DevOps team with two engineers:

- Engineer A executes `terraform apply`.
- Engineer B executes `terraform apply` against the same state at the same time.
- Without locking, concurrent operations may conflict and lead to inconsistent infrastructure or state.

State locking coordinates access to the shared state.

In our assignment:

- Amazon S3 stores the Terraform state file.
- DynamoDB provides state locking.
- Terraform automatically acquires and releases the lock during supported operations.

2. Prerequisites

Before starting Task 5, I completed the following:

Task 3: Created an EC2 instance with Apache HTTPD using Terraform.

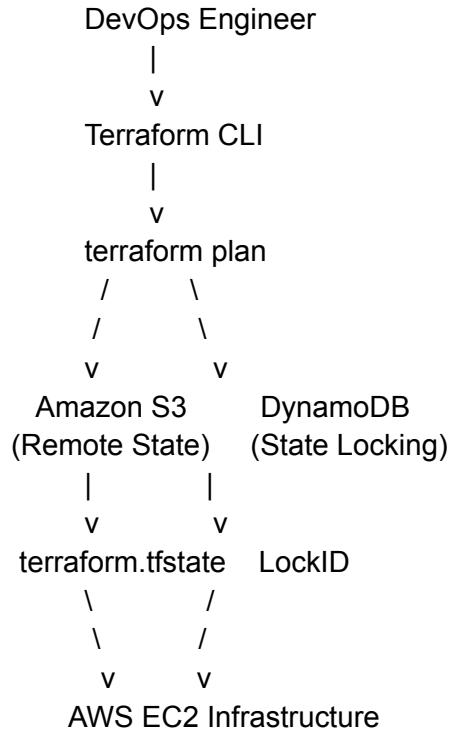
Task 4: Configured Amazon S3 as the Terraform remote backend and successfully migrated the EC2 state file.

Existing infrastructure

Component	Details
AWS Region	us-east-1
S3 bucket	`terraform05-986881030460-fb58b087846653c9a85ffb6795`

Terraform state key	`terraform05/task3/terraform.tfstate`
DynamoDB table	`terraform05-state-lock-986881030460`
AWS CLI profile	`terraform-lab`
EC2 Terraform project	`task-3-httpd`
Backend bootstrap project	`task-4-backend-bootstrap`

3. Architecture



Workflow

1. Terraform connects to the configured S3 backend.
2. Terraform acquires a state lock through DynamoDB.
3. Terraform reads the current state from S3.
4. Terraform checks the configuration against actual AWS infrastructure.
5. Terraform performs the required operation.
6. Updated state is stored in S3 when necessary.
7. Terraform releases the state lock.

4. Open the Backend Bootstrap Project

Opened Windows PowerShell and navigated to:

cd C:\Users\HP\terraform-05\task-4-backend-bootstrap

Verified the existing Terraform resources:

terraform state list

The bootstrap project already managed:

data.aws_caller_identity.current
aws_s3_bucket.terraform_state
aws_s3_bucket_public_access_block.state_security
aws_s3_bucket_server_side_encryption_configuration.state_encryption
aws_s3_bucket_versioning.state_versioning

The backend bootstrap project was maintained separately from the EC2 project to avoid accidentally destroying the state-storage infrastructure.

5. Create the DynamoDB Table Using Terraform

Opened the existing `main.tf` file:

code main.tf

Added the following resource without deleting the existing S3 configuration:

```
# Terraform-05 Task 5 - DynamoDB Locking

resource "aws_dynamodb_table" "terraform_lock" {
  name         = "terraform05-state-lock-986881030460"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  tags = {
    Name    = "Terraform05-State-Lock"
    Project = "Terraform-05"
  }
```

```
}

output "dynamodb_table_name" {
  value = aws_dynamodb_table.terraform_lock.name
}
```

Code explanation

Attribute	Purpose
`aws_dynamodb_table`	Creates a DynamoDB table
`terraform_lock`	Terraform logical resource name
`name`	Actual DynamoDB table name
`billing_mode`	Uses on-demand billing
`hash_key`	Defines the partition key
`LockID`	Required partition key for Terraform DynamoDB locking
`type = "S"`	String data type
`tags`	Identifies the AWS resource
`output`	Displays the created table name

`PAY_PER_REQUEST` was selected for this lab instead of manually configuring provisioned read and write capacity.

6. Initialize and Validate Terraform

Configured the AWS CLI profile:

$env:AWS_PROFILE = "terraform-lab"

Ran the Terraform commands:

terraform init
terraform fmt
terraform validate

Then checked the execution plan:

terraform plan

Actual Terraform Plan Result

```
# aws_dynamodb_table.terraform_lock will be created

+ resource "aws_dynamodb_table" "terraform_lock" {
    + billing_mode = "PAY_PER_REQUEST"
    + hash_key     = "LockID"
    + name         = "terraform05-state-lock-986881030460"

    + attribute {
        + name = "LockID"
        + type = "S"
      }
  }
```

Plan: 1 to add, 0 to change, 0 to destroy.

Result: The plan showed one DynamoDB table to be created, with no modifications or deletions to the existing S3 resources.

7. Apply the DynamoDB Configuration

Executed:

terraform apply

Reviewed and approved the Terraform plan.

Terraform created the DynamoDB resource and recorded it in the bootstrap project's state.

The table's name was configured as:

terraform05-state-lock-986881030460

Verify the Terraform Output

Executed:

terraform output

Actual output from subsequent verification:

dynamodb_table_name = "terraform05-state-lock-986881030460"

state_bucket_name = "terraform05-986881030460-fb58b087846653c9a85ffb6795"

8. Verify the DynamoDB Resource

Executed the following command in the bootstrap folder:

terraform apply

Actual Output

aws_dynamodb_table.terraform_lock: Refreshing state...
[id=terraform05-state-lock-986881030460]

No changes. Your infrastructure matches the configuration.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

This confirmed that Terraform recognized the existing DynamoDB table and did not need to recreate or modify it.

Additional AWS CLI Verification

The table can also be checked with:

```
aws dynamodb describe-table `
  --table-name terraform05-state-lock-986881030460 `
  --region us-east-1 `
  --profile terraform-lab `
  --query "Table.{Name:TableName,Status:TableStatus,KeySchema:KeySchema}" `
  --output json
```

Expected table status:

ACTIVE

This is an additional verification command; its actual AWS CLI output was not included in the recorded practical.

9. Configure DynamoDB Locking in the S3 Backend

Navigated to the EC2 Terraform project:

cd C:\Users\HP\terraform-05\task-3-httpd

Opened the existing backend configuration:

notepad backend.tf

Updated the file as follows:

```
terraform {
  backend "s3" {
    bucket         = "terraform05-986881030460-fb58b087846653c9a85ffb6795"
    key            = "terraform05/task3/terraform.tfstate"
    region         = "us-east-1"
    profile        = "terraform-lab"
    encrypt        = true
    dynamodb_table = "terraform05-state-lock-986881030460"
  }
}
```

Backend configuration explanation

Attribute	Purpose
`backend "s3"`	Configures S3 remote state storage
`bucket`	Identifies the existing state bucket
`key`	Identifies the Terraform state object
`region`	AWS region
`profile`	AWS credentials profile
`encrypt`	Requests encryption of the state object
`dynamodb_table`	Enables DynamoDB-based locking using the specified table

The S3 bucket and state key were not changed. Only the DynamoDB locking configuration was added.

10. Reinitialize the Terraform Backend

After modifying `backend.tf`, executed:

terraform init -reconfigure

Why was `-reconfigure` required?

We updated the settings for our existing S3 backend.

`terraform init -reconfigure` tells Terraform to initialize the backend again using the current configuration without automatically migrating state from the previously configured backend.

11. Verify State Locking

Executed the following command inside `task-3-httpd`:

terraform plan -lock-timeout=30s

Actual Output

aws_instance.httpd_server: Refreshing state...
[id=i-0de27553fcc7fde66]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure
against your configuration and found no
differences, so no changes are needed.

Releasing state lock. This may take a few moments...

Result

State locking verification: PASSED

The Terraform plan finished successfully and displayed the state-lock release message.

This confirms that the locking-enabled backend was functioning during the operation.

12. Verify the DynamoDB Backend Configuration

Executed:

$meta = Get-Content .\.terraform\terraform.tfstate -Raw | ConvertFrom-Json

Then:

$meta.backend.config.dynamodb_table

Actual Output

terraform05-state-lock-986881030460

This confirms that Terraform's initialized backend configuration references the correct DynamoDB table.

13. Troubleshooting Encountered

Issue 1 — Backend Initialization Required

Error:

Error: Backend initialization required,
please run "terraform init"

Reason: Terraform detected backend configuration changes or initialization metadata requiring an update.

Resolution: Checked the backend configuration and reinitialized Terraform appropriately.

The backend bootstrap project was kept separate from the EC2 project's S3 remote backend.

Issue 2 — Local Backend Metadata File Missing

Error:

Cannot find path
.terraform\terraform.tfstate
because it does not exist.

Explanation: When Terraform uses its default local backend, the infrastructure state is normally stored in the root directory as `terraform.tfstate`. A `.terraform\terraform.tfstate` backend metadata file is not always present.

Verification:

Test-Path .\terraform.tfstate
terraform state list

The bootstrap project's managed resources were successfully listed.

Issue 3 — Verifying That Locking Was Configured

Running `terraform plan` alone does not prove which locking method is configured.

Resolution: Checked the initialized backend metadata and confirmed:

dynamodb_table = terraform05-state-lock-986881030460

Also observed Terraform's state-lock release message during a successful plan.

14. Real-Time DevOps Scenario

Suppose two DevOps engineers share the same Terraform configuration and S3 state.

Engineer A begins a Terraform operation.

Terraform acquires a lock using the DynamoDB table.

While the state is locked, Engineer B attempts another Terraform operation against the same state.

Engineer B cannot acquire the same lock immediately. Depending on the command and its lock timeout, Terraform waits or returns a state-lock error.

After Engineer A completes the operation, Terraform releases the lock, allowing another operation to proceed.

This prevents unsafe simultaneous state modifications.

15. Important Interview Questions and Answers

Q1. What is Terraform state locking?

Terraform state locking is a mechanism that coordinates access to a shared state file so multiple operations do not modify it at the same time.

Q2. Why did we use DynamoDB?

In this assignment, DynamoDB was used as a locking mechanism for Terraform state stored in Amazon S3.

Q3. What is the difference between S3 and DynamoDB in this task?

S3 stores the Terraform state file. DynamoDB coordinates locking of that state during Terraform operations.

Q4. Why is `LockID` required?

The legacy S3 backend's DynamoDB locking mechanism requires a string partition key named `LockID` to manage lock records.

Q5. What does `PAY_PER_REQUEST` mean?

It is DynamoDB's on-demand billing mode. Capacity adjusts according to requests without requiring manually configured read and write capacity units.

Q6. What does `terraform init -reconfigure` do?

It reinitializes Terraform using the currently defined backend configuration without automatically migrating the previous backend state.

Q7. What happens if two users execute Terraform simultaneously?

When both operations require the same state lock, one can acquire it while the other must wait or fail to acquire it, depending on the timeout configuration.

Q8. What is the purpose of `-lock-timeout=30s`?

It tells Terraform to wait up to 30 seconds to acquire a state lock rather than immediately failing when the lock is unavailable.

Q9. How did you verify DynamoDB locking?

I configured the `dynamodb_table` attribute in the S3 backend, initialized Terraform, ran `terraform plan -lock-timeout=30s`, observed the state-lock release message, and verified the DynamoDB table name in backend metadata.

Q10. Is DynamoDB locking recommended for new Terraform projects?

No. DynamoDB locking is deprecated. Modern Terraform supports native S3 locking using `use_lockfile = true`. We used DynamoDB to complete the institute's assigned practical.

16. Final Verification Summary

Verification	Result
DynamoDB resource managed by Terraform	Passed
DynamoDB table name	Verified
S3 backend configured	Passed
DynamoDB locking configured	Passed

Terraform plan	No changes
State lock released successfully	Passed
EC2 state preserved	Passed

Final Result

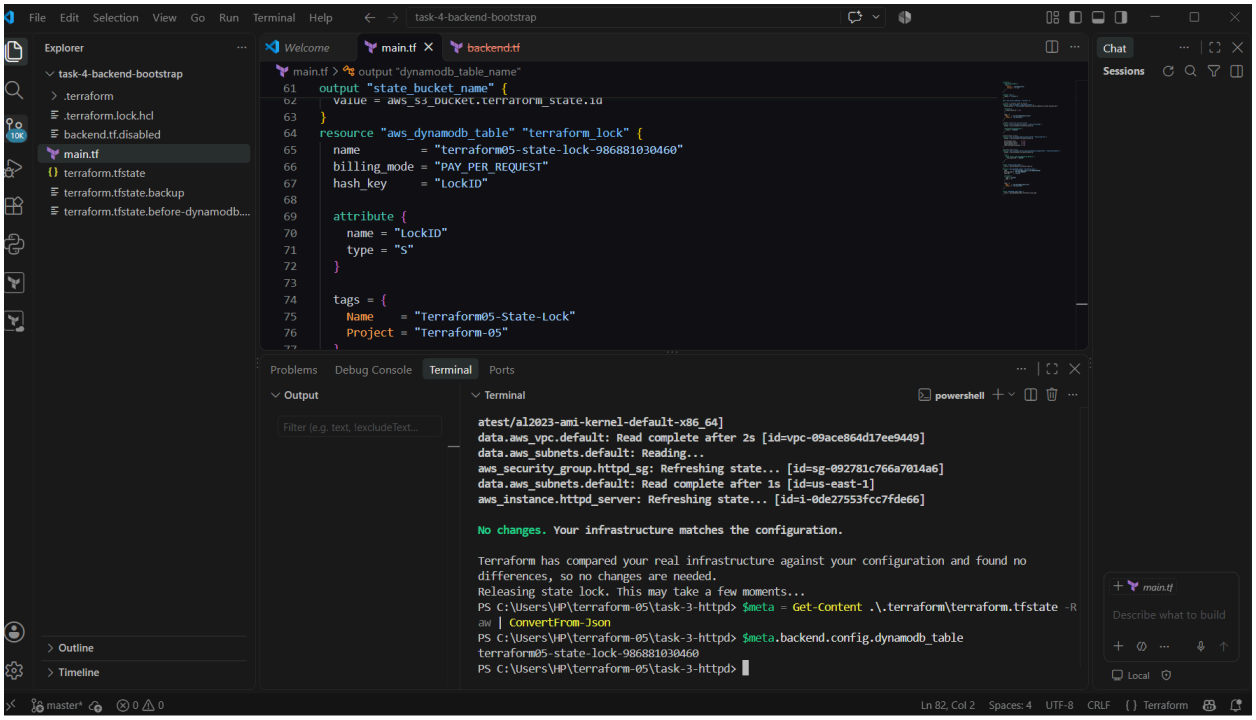
Terraform-05 Task 5 — Successfully Completed

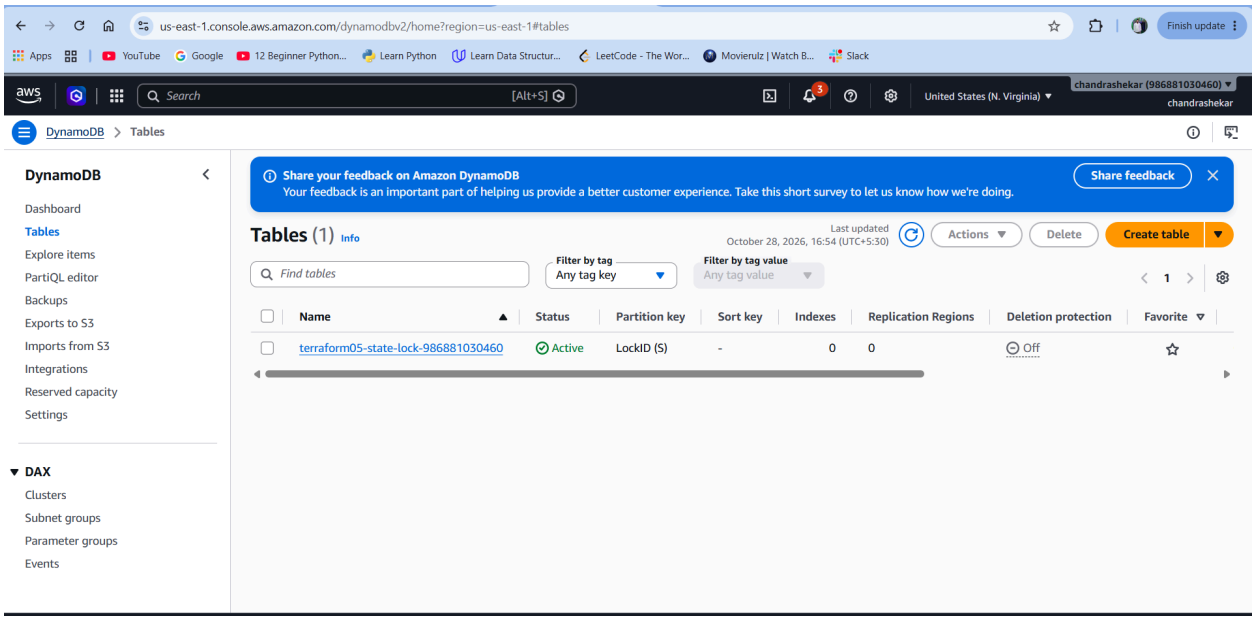
Created an Amazon DynamoDB table using Terraform and configured it as the locking mechanism for the existing S3 remote backend.

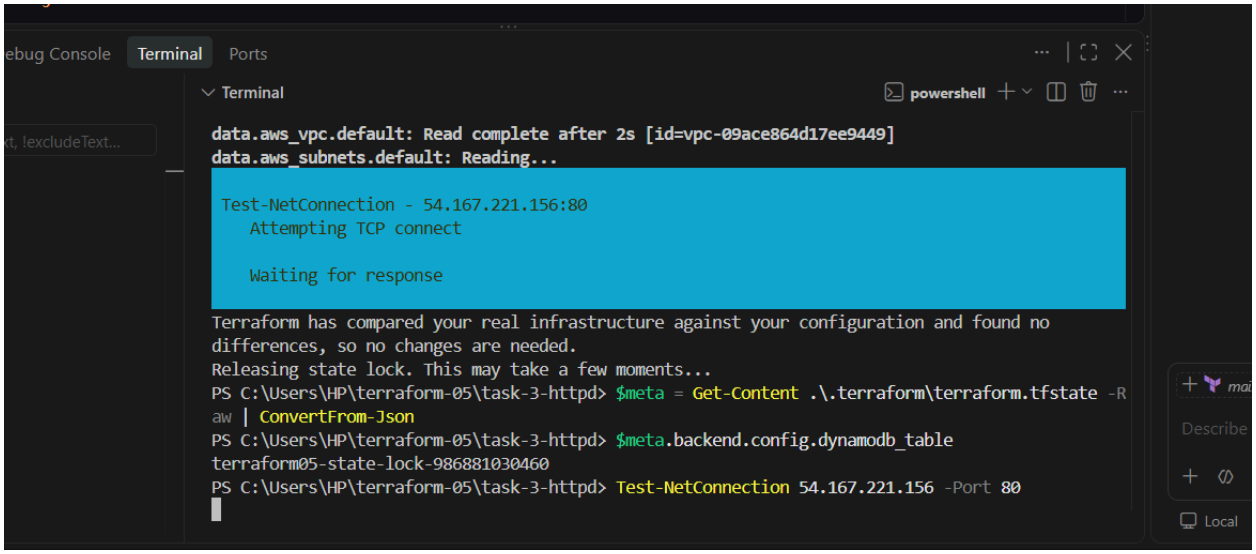
Verified the backend configuration and successful state-lock release during a Terraform operation.

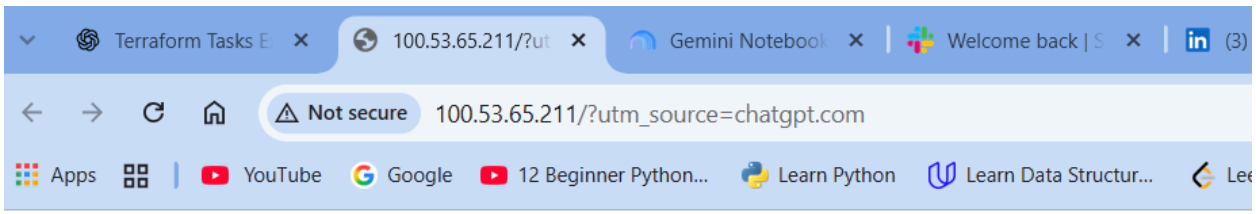
Learning outcome: Understood the purpose of Terraform state locking, how DynamoDB-based locking coordinates access to S3 remote state, and how to verify the locking configuration through Terraform commands.

Important: Do not delete the S3 state bucket or DynamoDB table while Terraform projects still depend on them. Always complete infrastructure cleanup and state migration before removing backend resources.


File Edit Selection View Go Run Terminal Help
task-4-backend-bootstrap
Explorer
task-4-backend-bootstrap
.terraform
.terraform.lock.hcl
backend.tf.disabled
main.tf
terraform.tfstate
terraform.tfstate.backup
terraform.tfstate.before-dynamodb....
Outline
Timeline
Welcome
main.tf
backend.tf
main.tf > output "dynamodb_table_name"
61 output "state_bucket_name" {
62 value = aws_s3_bucket.terraform_state.id
63 }
64 resource "aws_dynamodb_table" "terraform_lock" {
65 name = "terraform05-state-lock-986881030460"
66 billing_mode = "PAY_PER_REQUEST"
67 hash_key = "LockID"
68
69 attribute {
70 name = "LockID"
71 type = "S"
72 }
73
74 tags = {
75 Name = "Terraform05-State-Lock"
76 Project = "Terraform-05"
Problems
Debug Console
Terminal
Ports
Output
Filter (e.g. text, !exclude, \text...
Terminal
powershell
atest/al2023-ami-kernel-default-x86_64]
data.aws_vpc.default: Read complete after 2s [id=vpc-09ace864d17ee9449]
data.aws_subnets.default: Reading...
aws_security_group.httpd_sg: Refreshing state... [id=sg-092781c766a7014a6]
data.aws_subnets.default: Read complete after 1s [id=us-east-1]
aws_instance.httpd_server: Refreshing state... [id=i-0de27553fcc7fde66]
No changes. Your infrastructure matches the configuration.
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
Releasing state lock. This may take a few moments...
PS C:\Users\HP\terraform-05\task-3-httpd> $meta = Get-Content .\.terraform\terraform.tfstate -Raw | ConvertFrom-Json
PS C:\Users\HP\terraform-05\task-3-httpd> $meta.backend.config.dynamodb_table
terraform05-state-lock-986881030460
PS C:\Users\HP\terraform-05\task-3-httpd>
Chat
Sessions
main.tf
Describe what to build
Local
master*
Ln 82, Col 2
Spaces: 4
UTF-8
CRLF
Terraform


us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#tables
Finish update
Apps
YouTube
Google
12 Beginner Python...
Learn Python
Learn Data Structur...
LeetCode - The Wor...
Movierulz | Watch B...
Slack
aws
Search
[Alt+S]
United States (N. Virginia)
chandrashekar (986881030460)
chandrashekar
DynamoDB > Tables
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
Share your feedback on Amazon DynamoDB
Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.
Share feedback
Tables (1) Info
Last updated
October 28, 2026, 16:54 (UTC+5:30)
Actions
Delete
Create table
Find tables
Filter by tag
Any tag key
Filter by tag value
Any tag value
Name
Status
Partition key
Sort key
Indexes
Replication Regions
Deletion protection
Favorite
terraform05-state-lock-986881030460
Active
LockID (S)
-
0
0
Off





Terraform-05 HTTPD Server Working!